

BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN
MÔN HỌC: PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG
(MAD)

HỆ THỐNG PODY
PODCAST AI PLATFORM

Đề tài: Xây dựng ứng dụng di động Podcast cá nhân hóa ứng dụng Trí tuệ nhân tạo (Pody)

Nhóm QLĐT (Nhóm môn học): Nhóm 01

Nhóm BTL: Nhóm 01

Danh sách thành viên:

- Phan Thanh Tùng - MSSV: B21DCCN999
- Nguyễn Văn A - MSSV: B21DCCN001
- Trần Thị B - MSSV: B21DCCN002
- Phạm Văn C - MSSV: B21DCCN003

Sinh viên thực hiện báo cáo cá nhân: Phan Thanh Tùng
(B21DCCN999)

Hà Nội, Năm 2026

Bảng đóng góp của thành viên

Dưới đây là chi tiết phân chia công việc và mức độ đóng góp của từng thành viên trong nhóm 01 đối với Bài Tập Lớn môn học Phát triển Ứng dụng Di động:

STT	Họ và Tên	MSSV	Phạm vi phụ trách	Công việc cụ thể	Đóng góp
1	Phan Thanh Tùng	B21DCCN999	AI tạo podcast (ai-service, content-service)	Nhóm trưởng, thiết kế hệ thống, sinh kịch bản đối thoại đa MC ảo và TTS, quản lý show/episode, viết báo cáo.	25%
2	Nguyễn Văn A	B21DCCN001	Báo và podcast báo (article_service, embedding_service)	Tích hợp cào tin tự động từ RSS, chunking bài báo, tạo embedding vector và semantic search.	25%
3	Trần Thị B	B21DCCN002	Hạ tầng và xác thực (api-gateway, identity-service)	Phát triển Gateway, JWT auth middleware, xác thực Google Sign-in, Outbox pattern.	25%
4	Phạm Văn C	B21DCCN003	Thông báo (notification-service)	Nhận thông báo in-app, unread count, email delivery logs, consumer Kafka retry và DLQ.	25%

Bảng 1: Bảng phân chia công việc và đóng góp thành viên

Mục lục

Bảng đóng góp của thành viên	1
Danh sách từ viết tắt	6
1 Giới thiệu ứng dụng và Phân tích yêu cầu	7
1.1 Giới thiệu về ứng dụng Pody	7
1.2 Lý do chọn đề tài	7
1.3 Concept thực hiện ứng dụng	8
1.3.1 Luồng Listener (Người nghe thông tin)	8
1.3.2 Luồng Creator (Nhà sáng tạo nội dung)	8
1.4 Phân tích yêu cầu ứng dụng	9
1.4.1 Yêu cầu chức năng	9
1.4.2 Yêu cầu phi chức năng	10
1.5 Phân tích và lựa chọn công nghệ sử dụng	10
1.5.1 Ứng dụng di động (Frontend Mobile)	11
1.5.2 Cửa ngõ API Gateway và Dịch vụ cốt lõi	11
1.5.3 Dịch vụ xử lý AI và Tin tức	11
1.5.4 Cơ sở dữ liệu và Lưu trữ	11
1.5.5 Hàng đợi sự kiện và CDC (Change Data Capture)	11
1.5.6 Bộ nhớ đệm và Trình cung cấp AI	12
2 Phân tích và Thiết kế hệ thống	13
2.1 Kiến trúc tổng quan hệ thống	13
2.2 Biểu đồ Use Case	13
2.2.1 Biểu đồ Use Case tổng quan hệ thống (Mức nhóm)	13
2.2.2 Biểu đồ Use Case chi tiết - Luồng xử lý AI Podcast (Mức cá nhân - Phan Thanh Tùng)	13
2.3 Thiết kế sơ đồ thực thể quan hệ (ERD) chi tiết	14
2.3.1 Cấu trúc bảng của Dịch vụ định danh (identity-service)	14
2.3.2 Cấu trúc bảng của Dịch vụ nội dung (content-service)	16
2.3.3 Cấu trúc bảng của Dịch vụ Tin tức (article-service)	18
2.4 Chi tiết Bảng mô tả cơ sở dữ liệu vật lý	19
2.4.1 Bảng <code>users</code> (identity_service)	19
2.4.2 Bảng <code>shows</code> (content_service)	19
2.4.3 Bảng <code>episodes</code> (content_service)	19
2.5 Cơ chế xác thực phân tán bằng JWT Middleware	20
2.6 Mô hình xử lý và phát sinh kịch bản Stub Fallback	21
2.7 Phân tích cơ chế Outbox Pattern và CDC Debezium	22
2.7.1 Cơ chế hoạt động của Outbox Pattern	22
2.7.2 Cơ chế Change Data Capture (CDC) với Debezium	23

2.8	Biểu đồ lớp (Class Diagram)	23
2.9	Biểu đồ tuần tự (Sequence Diagram)	23
2.10	Thiết kế giao diện đáp ứng chức năng	23
2.10.1	Màn hình trang chủ (Home Screen)	23
2.10.2	Màn hình trình phát âm thanh đồng bộ (Player Screen)	23
3	Kết quả triển khai và Thử nghiệm	27
3.1	Mô hình triển khai hệ thống	27
3.1.1	Cấu hình tệp tin Docker Compose (docker-compose.yml)	27
3.2	Các bước cài đặt và triển khai	29
3.3	Kết quả các tính năng chính trên giao diện	30
3.4	Kết quả thử nghiệm và kiểm thử tự động	30
3.4.1	Số liệu thống kê vận hành hệ thống trong giai đoạn kiểm thử	30
3.5	Kết luận, hạn chế và định hướng nâng cấp	30
3.5.1	Kết luận	30
3.5.2	Hạn chế	31
3.5.3	Định hướng nâng cấp	31
	Tài liệu tham khảo	32

Danh sách bảng

1	Bảng phân chia công việc và đóng góp thành viên	1
3.1	Bảng thống kê số liệu vận hành và dữ liệu kiểm thử	31

Danh sách hình vẽ

2.1	Biểu đồ Use Case tổng quan hệ thống	13
2.2	Biểu đồ Use Case chi tiết luồng xử lý AI Podcast	14
2.3	Sơ đồ thực thể quan hệ (ERD) hệ thống Pody	14
2.4	Biểu đồ lớp các thực thể cốt lõi hệ thống Pody	24
2.5	Sơ đồ tuần tự luồng tạo AI Podcast từ bài báo	25
2.6	Phác thảo giao diện Màn hình chính (Home Screen)	25
2.7	Phác thảo giao diện Trình phát nhạc đồng bộ chữ (Player Screen)	26

Danh sách từ viết tắt

Từ viết tắt	Ý nghĩa đầy đủ
AI	Artificial Intelligence (Trí tuệ nhân tạo)
TTS	Text To Speech (Chuyển đổi văn bản thành giọng nói)
CDC	Change Data Capture (Ghi nhận sự thay đổi dữ liệu cơ sở dữ liệu)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
JWT	JSON Web Token (Mã xác thực dạng chuỗi ký số JSON)
ERD	Entity Relationship Diagram (Sơ đồ thực thể quan hệ)
RDBMS	Relational Database Management System (Hệ quản trị CSDL quan hệ)
FCM	Firebase Cloud Messaging (Dịch vụ gửi thông báo đẩy của Firebase)
S3	Simple Storage Service (Chuẩn lưu trữ đối tượng đám mây)
SMTP	Simple Mail Transfer Protocol (Giao thức truyền tải thư tín đơn giản)
LLM	Large Language Model (Mô hình ngôn ngữ lớn)

Chương 1

Giới thiệu ứng dụng và Phân tích yêu cầu

1.1 Giới thiệu về ứng dụng Pody

Hệ thống **Pody** là một nền tảng nghe và tự động sản xuất podcast thể hệ mới, tích hợp sâu công nghệ Trí tuệ nhân tạo (Podcast AI Platform) nhằm mang lại trải nghiệm âm thanh cá nhân hóa vượt trội. Điểm khác biệt lớn nhất của Pody so với các nền tảng podcast truyền thống là khả năng chuyển đổi trực tiếp văn bản (tin tức báo chí, kịch bản hội thoại) thành các tệp âm thanh podcast sinh động nhờ MC ảo và giọng đọc nhân tạo.

Đối với người nghe (Listener), Pody là một ứng dụng di động thông minh cho phép họ không chỉ thưởng thức các nội dung podcast thông thường, mà còn có thể đọc báo và yêu cầu hệ thống tự động tóm tắt tin tức để nghe dưới dạng podcast tin tức ngắn (AI News Digest Podcast). Đối với nhà sáng tạo (Creator), Pody cung cấp một bộ công cụ hỗ trợ toàn diện từ khâu lập kế hoạch sản xuất (Production Plan), sinh kịch bản đối thoại đa nhân vật tự động bằng Mô hình ngôn ngữ lớn (LLM), cho đến khâu sản xuất tệp tin âm thanh đi kèm phụ đề chạy chữ được đồng bộ hóa chuẩn xác ở cấp độ từ ngữ (Word-level Aligned Transcript).

Hệ thống được thiết kế và triển khai trên nền tảng kiến trúc Microservices phân tán với các dịch vụ độc lập, giao tiếp thông qua API Gateway và hệ thống hàng đợi sự kiện Apache Kafka, kết hợp với ứng dụng di động Flutter mang lại hiệu năng cao và khả năng mở rộng tối ưu.

1.2 Lý do chọn đề tài

Trong thời đại bùng nổ thông tin số, khối lượng nội dung dạng văn bản (tin tức, bài phân tích, tài liệu học tập) tăng trưởng nhanh chóng mỗi ngày, khiến con người gặp nhiều khó khăn trong việc dành thời gian tập trung đọc hiểu. Đồng thời, xu hướng tiêu thụ nội dung âm thanh (Audio format) đang trở nên vô cùng ưu việt và phổ biến do tính linh hoạt cao, giúp người dùng tận dụng thời gian rảnh rỗi khi đang lái xe, tập thể dục, dọn dẹp nhà cửa để tiếp cận tri thức.

Tuy nhiên, thị trường podcast hiện nay đang tồn tại hai thách thức lớn:

- **Đối với người nghe:** Nội dung các kênh podcast mang tính chất đại trà, chưa được cá nhân hóa hoặc cập nhật theo đúng sở thích và nhu cầu tức thời của từng cá nhân tại thời điểm nghe.
- **Đối với nhà sáng tạo nội dung:** Quy trình sản xuất một tập podcast truyền thống đòi hỏi đầu tư rất lớn về thiết bị thu âm chất lượng cao, phòng thu tiêu âm,

kỹ năng viết kịch bản hấp dẫn, cũng như chi phí và thời gian biên tập, hậu kỳ âm thanh phức tạp.

Xuất phát từ thực tiễn đó, nhóm nghiên cứu lựa chọn đề tài xây dựng ứng dụng **Pody** nhằm kết hợp sức mạnh của thiết bị di động với Trí tuệ nhân tạo tạo sinh (Generative AI). Đề tài này không chỉ giải quyết bài toán tóm tắt tin tức cá nhân hóa cho người dùng mà còn tối ưu hóa quy trình làm podcast cho các nhà sáng tạo thông qua các MC ảo. Về mặt kỹ thuật, đề tài cung cấp một cơ hội lý tưởng để áp dụng các công nghệ phát triển ứng dụng di động (Flutter), kiến trúc Microservices (Go, Python FastAPI), lưu trữ và truy vấn vector (PostgreSQL với pgvector), cơ chế đồng bộ hướng sự kiện (Kafka, Debezium, Outbox Pattern) và hệ thống thông báo thời gian thực.

1.3 Concept thực hiện ứng dụng

Triết lý và concept cốt lõi của ứng dụng Pody được gói gọn trong khẩu hiệu: **"Đọc nhanh, nghe sau, tạo podcast bằng AI"**. Hệ thống được thiết kế xoay quanh hai luồng nghiệp vụ chính tương ứng với hai đối tượng người dùng mục tiêu:

1.3.1 Luồng Listener (Người nghe thông tin)

Luồng Listener hướng tới việc cá nhân hóa trải nghiệm tiêu thụ tin tức hàng ngày của người dùng:

- **Khám phá nội dung:** Người dùng có thể tìm kiếm, đọc các bài báo mới nhất được cào tự động từ các nguồn tin uy tín, hoặc duyệt danh sách các Show podcast truyền thống.
- **Nghe tin tức cá nhân hóa:** Người dùng lựa chọn một nhóm bài báo mà mình quan tâm, sau đó yêu cầu hệ thống tự động biên tập và sinh podcast tin tức ngắn (AI News Digest Podcast).
- **Trình phát nhạc thông minh:** Hỗ trợ phát tệp âm thanh đồng thời hiển thị kịch bản chữ chạy tự động cuộn (Auto-scroll) và tô sáng (Highlight) từ ngữ tương ứng với giọng nói của MC ảo.
- **Lưu trữ và Tương tác:** Cho phép đánh dấu (Bookmark) các tập podcast yêu thích, bình luận và bày tỏ cảm xúc trên các bài báo/tập podcast, đồng thời nhận các thông báo mới từ hệ thống.

1.3.2 Luồng Creator (Nhà sáng tạo nội dung)

Luồng Creator tập trung vào việc đơn giản hóa quy trình sản xuất nội dung số bằng AI:

- **Lên ý tưởng và Lập kế hoạch:** Creator trò chuyện với Trí tuệ nhân tạo (AI Chat-create) để tìm kiếm ý tưởng, tinh chỉnh chủ đề và sinh kế hoạch sản xuất tập podcast (Production Plan).
- **MC ảo đa giọng đọc (Multi-host AI):** Thiết lập danh sách các MC ảo tham gia dẫn chương trình với các cấu hình giọng đọc khác nhau (Voice Profiles).
- **Sinh kịch bản và Âm thanh tự động:** Hệ thống tự động sinh kịch bản hội thoại đối đáp tự nhiên giữa các MC ảo, chuyển đổi kịch bản thành giọng nói tiếng Việt chất lượng cao (TTS), tự động ghép nối và xuất bản tập podcast hoàn chỉnh kèm theo tệp tin transcript đồng bộ.

1.4 Phân tích yêu cầu ứng dụng

1.4.1 Yêu cầu chức năng

Hệ thống Pody phải đáp ứng đầy đủ các nhóm chức năng nghiệp vụ sau:

1. Xác thực và Quản lý người dùng:

- Đăng ký tài khoản mới bằng địa chỉ email.
- Xác minh tài khoản thông qua mã kích hoạt được gửi tự động qua email (Email Verification).
- Đăng nhập bằng tài khoản email mật khẩu truyền thống hoặc đăng nhập nhanh bằng tài khoản Google (Google Sign-In).
- Quên mật khẩu và đặt lại mật khẩu mới thông qua mã OTP bảo mật gửi qua email.
- Xem và cập nhật thông tin cá nhân (tên hiển thị, ảnh đại diện, tiểu sử).

2. Quản lý và Duyệt Podcast:

- Xem danh sách các Show podcast và các danh mục phân loại.
- Xem thông tin chi tiết của từng Show và danh sách các tập phát sóng (Episodes) trực thuộc.
- Xem chi tiết một tập podcast, hiển thị thông tin MC, thời lượng và phát âm thanh.
- Đăng ký/theo dõi (Subscribe) các kênh podcast ưa thích và lưu các tập podcast vào danh sách yêu thích (Bookmark).

3. Quản lý và Tương tác Tin tức:

- Đọc nội dung các bài báo được cào tự động và cập nhật liên tục từ các nguồn tin RSS.
- Lọc và tìm kiếm các bài viết theo từ khóa hoặc theo danh mục chủ đề.
- Thực hiện bày tỏ cảm xúc (Reaction - Like/Love) và viết bình luận (Comment) dưới mỗi bài viết.
- Thu thập dữ liệu hành vi đọc của người dùng để đo lường các chỉ số tương tác (Metrics).

4. Luồng tạo Podcast tin tức từ AI (AI Podcast Flow):

- Cho phép người dùng chọn nhiều bài báo cùng lúc để gửi yêu cầu sinh podcast.
- Cấu hình các tùy chọn cho Job: lựa chọn giọng đọc AI (Voice Profile), thời lượng tóm tắt mong muốn (Target minutes), ngôn ngữ.
- Theo dõi trạng thái tiến độ xử lý của Job thông qua các trạng thái tuần tự bất đồng bộ: `queued` → `researching` → `drafting` → `validating` → `synthesizing` → `uploading` → `completed` (hoặc `failed`).
- Cho phép phát tệp âm thanh wav kết quả kèm theo transcript JSON đồng bộ.

5. Luồng tạo Show bằng AI (AI Show Creator):

- Creator thiết lập thông tin Show, ảnh bìa và cấu hình các MC ảo.

- Khởi tạo cuộc trò chuyện với AI dưới dạng Thread để sinh kế hoạch sản xuất (Production Plan).
- Tạo tập phát sóng (Episode) mới tự động từ Production Plan đã được AI duyệt.

6. Hệ thống thông báo (Notification System):

- Xem danh sách thông báo in-app trong hộp thư cá nhân (Notification Inbox) kèm theo đếm số lượng tin nhắn chưa đọc (Unread Count).
- Đánh dấu các thông báo đã đọc (Mark as read) hoặc đọc tất cả (Mark all as read).
- Điều chỉnh cài đặt nhận thông báo (Notification Settings) trong tài khoản.
- Gửi các email hệ thống tự động (OTP xác minh, mật khẩu mới, thông báo Job hoàn thành).

1.4.2 Yêu cầu phi chức năng

Để hệ thống vận hành trơn tru và đạt chất lượng tốt nhất, các yêu cầu phi chức năng sau cần được tuân thủ nghiêm ngặt:

1. **Kiến trúc Microservices độc lập:** Hệ thống được chia tách thành các dịch vụ độc lập theo miền nghiệp vụ để dễ dàng phát triển song song, bảo trì và tự động mở rộng (scale) độc lập các dịch vụ chịu tải nặng như dịch vụ xử lý AI hay tóm tắt tin tức.
2. **Cổng kết nối API Gateway:** Toàn bộ request từ phía Client phải đi qua API Gateway điều phối. Gateway chịu trách nhiệm định tuyến (Routing), kiểm soát CORS, log thông tin và áp dụng Middleware kiểm tra tính hợp lệ của mã JWT trước khi chuyển tiếp.
3. **Độc lập dữ liệu và Cơ sở dữ liệu riêng:** Mỗi microservice sở hữu một cơ sở dữ liệu vật lý riêng biệt. Hệ thống không sử dụng các liên kết khóa ngoại vật lý (Foreign Keys) chéo giữa cơ sở dữ liệu của các dịch vụ để tránh sự phụ thuộc chặt chẽ (tight coupling).
4. **Khả năng tích hợp và Tài liệu hóa:** Các dịch vụ phải có đầy đủ tài liệu đặc tả API chuẩn OpenAPI/Swagger để các dịch vụ khác dễ dàng tích hợp. Phải cung cấp API kiểm tra sức khỏe hệ thống (Health Check) để giám sát trạng thái hoạt động của từng dịch vụ.
5. **Kiểm thử và Đóng gói:** Đạt độ bao phủ kiểm thử (test coverage) tốt ở các phần logic nghiệp vụ lõi. Toàn bộ hệ thống backend phải được đóng gói gọn gàng thông qua các container Docker và được quản lý tập trung bằng Docker Compose để dễ dàng triển khai demo ở môi trường local.

1.5 Phân tích và lựa chọn công nghệ sử dụng

Hệ thống Pody lựa chọn các công nghệ hiện đại, có độ ổn định và hiệu năng cao để giải quyết các bài toán kỹ thuật đặt ra:

1.5.1 Ứng dụng di động (Frontend Mobile)

- **Công nghệ lựa chọn:** Flutter (ngôn ngữ Dart).
- **Lý do lựa chọn:** Flutter cho phép phát triển ứng dụng di động đa nền tảng (iOS và Android) từ một cơ sở mã nguồn duy nhất, giúp tiết kiệm thời gian phát triển của nhóm. Công nghệ biên dịch trực tiếp ra mã máy của Flutter mang lại giao diện mượt mà (60-120fps), đáp ứng tốt các thành phần UI phức tạp như đĩa nhạc MC ảo xoay tròn và văn bản chạy chữ đồng bộ mili-giây với tiến trình âm thanh.

1.5.2 Cửa ngõ API Gateway và Dịch vụ cốt lõi

- **Công nghệ lựa chọn:** Ngôn ngữ Go (Golang) và thư viện định tuyến chi.
- **Lý do lựa chọn:** Go có hiệu năng thực thi cực kỳ nhanh, tiêu thụ rất ít bộ nhớ RAM và hỗ trợ xử lý đồng thời (Concurrency) tuyệt vời nhờ cơ chế Goroutines siêu nhẹ. Việc xây dựng `api-gateway`, `identity-service`, `content-service` bằng Go giúp hệ thống có khả năng chịu tải tốt và phản hồi các request thông tin lỗi với độ trễ thấp dưới 200ms.

1.5.3 Dịch vụ xử lý AI và Tin tức

- **Công nghệ lựa chọn:** Ngôn ngữ Python, web framework FastAPI và ORM SQLAlchemy.
- **Lý do lựa chọn:** Python sở hữu hệ sinh thái thư viện xử lý ngôn ngữ tự nhiên và Trí tuệ nhân tạo phong phú nhất hiện nay. FastAPI được chọn để phát triển các dịch vụ `ai-service`, `article-service`, và `embedding-service` nhờ hiệu năng xử lý bất đồng bộ (`async/await`) vượt trội, tốc độ phản hồi nhanh, và tự động tạo tài liệu kiểm thử đặc tả Swagger UI tự động.

1.5.4 Cơ sở dữ liệu và Lưu trữ

- **Công nghệ lựa chọn:** PostgreSQL, extension pgvector và MinIO Object Storage.
- **Lý do lựa chọn:** PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ vô cùng mạnh mẽ, đáng tin cậy và hỗ trợ các giao dịch ACID toàn diện. Đặc biệt, việc tích hợp extension `pgvector` cho phép lưu trữ trực tiếp các vector biểu diễn bài viết (768 chiều từ mô hình embedding) và thực hiện các câu lệnh tìm kiếm tương đồng ngữ nghĩa (Semantic Search) cực kỳ hiệu quả. MinIO được sử dụng làm kho lưu trữ đối tượng tương thích chuẩn S3 của Amazon để lưu trữ tệp âm thanh wav và tệp JSON transcript có dung lượng lớn.

1.5.5 Hàng đợi sự kiện và CDC (Change Data Capture)

- **Công nghệ lựa chọn:** Apache Kafka và Debezium Connector.
- **Lý do lựa chọn:** Kafka được sử dụng để xây dựng kiến trúc hướng sự kiện (Event-Driven Architecture) nhằm giảm sự phụ thuộc trực tiếp giữa các microservices (loose coupling). Debezium Connector được tích hợp để tự động chụp lại các thay đổi dữ liệu (Change Data Capture) từ PostgreSQL của dịch vụ tin tức, đẩy thẳng sự kiện

sang Kafka để dịch vụ embedding tiêu thụ, giúp đồng bộ hóa dữ liệu thời gian thực mà không cần viết code thủ công.

1.5.6 Bộ nhớ đệm và Trình cung cấp AI

- **Công nghệ lựa chọn:** Redis và Google Gemini / Text-to-Speech API.
- **Lý do lựa chọn:** Redis đóng vai trò lưu trữ cache các bài viết và rate-limiting của Gateway. Trí tuệ nhân tạo được cung cấp bởi Google Gemini API (**gemini-3.5-flash**) để tóm tắt văn bản và sinh kịch bản đối thoại chất lượng cao. API Google Cloud Text-to-Speech được sử dụng để chuyển đổi kịch bản chữ sang âm thanh nói tiếng Việt tự nhiên đa giọng đọc.

Chương 2

Phân tích và Thiết kế hệ thống

2.1 Kiến trúc tổng quan hệ thống

Hệ thống Pody được thiết kế theo kiến trúc Microservices phân rã theo miền nghiệp vụ:

- **Tầng Gateway (api-gateway):** Nhận request từ Client, giải mã token JWT, phân phối request đến microservice tương ứng.
- **Tầng Dịch vụ (Go/Python microservices):**
 - **identity-service:** Quản lý người dùng, ví tiền, gói hội viên.
 - **content-service:** Quản lý thư viện Show, Episode, MC ảo và Companion Blocks.
 - **notification-service:** Xử lý gửi email xác thực, OTP khôi phục mật khẩu.
 - **article-service:** Crawl bài viết, quản lý danh mục và lập lịch Job xử lý AI Podcast.
 - **embedding-service:** Xử lý chunking văn bản bài báo, gọi Gemini sinh embedding và đưa vào pgvector.
 - **ai-service:** Lên kế hoạch tập, viết kịch bản hội thoại đa AI Host.

2.2 Biểu đồ Use Case

2.2.1 Biểu đồ Use Case tổng quan hệ thống (Mức nhóm)

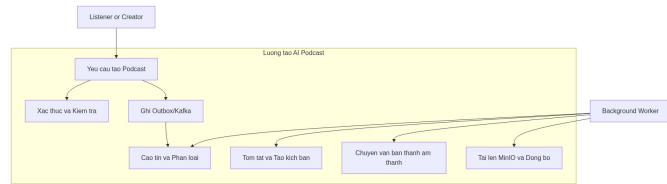
Hệ thống phân chia Use Case rõ ràng cho các tác nhân như mô tả trong Hình 2.1.



Hình 2.1: Biểu đồ Use Case tổng quan hệ thống

2.2.2 Biểu đồ Use Case chi tiết - Luồng xử lý AI Podcast (Mức cá nhân - Phan Thanh Tùng)

Phân tích chi tiết use case "Tạo AI News Podcast" do sinh viên Phan Thanh Tùng thiết kế và hiện thực ở phần Backend. Luồng này mô tả sự tương tác giữa Listener/Creator và hệ thống AI thông qua các bước bất đồng bộ.

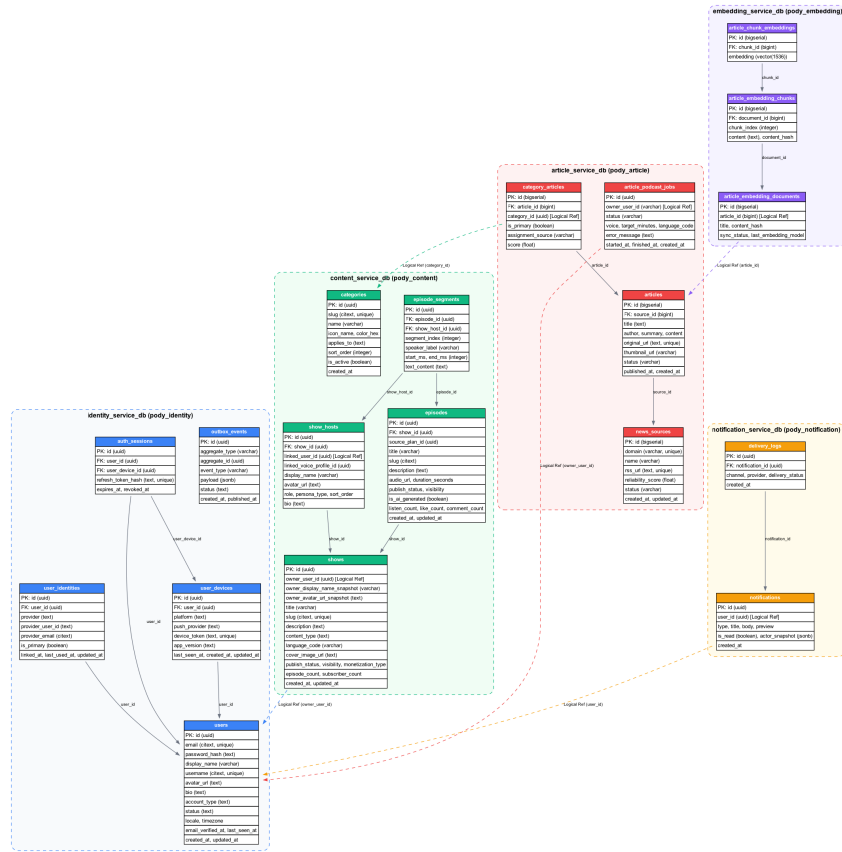


Hình 2.2: Biểu đồ Use Case chi tiết luồng xử lý AI Podcast

2.3 Thiết kế sơ đồ thực thể quan hệ (ERD) chi tiết

Cơ sở dữ liệu của Pody được thiết kế tách biệt theo từng service nhằm tối ưu hóa tính độc lập của kiến trúc Microservices. Dưới đây là sơ đồ thực thể quan hệ (ERD) tổng quan và chi tiết mã nguồn SQL DDL khởi tạo của các dịch vụ cốt lõi.

Sơ đồ thực thể quan hệ (ERD) tổng quan của hệ thống Pody được thể hiện ở Hình 2.3:



Hình 2.3: Sơ đồ thực thể quan hệ (ERD) hệ thống Pody

2.3.1 Cấu trúc bảng của Dịch vụ định danh (identity-service)

Dịch vụ định danh quản lý thông tin tài khoản người dùng, thiết bị kết nối để gửi thông báo đẩy (push notification) qua Firebase (FCM) và quản lý bảng Outbox để thực hiện Event-Driven.

```
-- 1. Bảng users
CREATE TABLE IF NOT EXISTS users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

email citext NOT NULL UNIQUE,
password_hash text,
display_name varchar(120) NOT NULL,
username citext UNIQUE,
avatar_url text,
bio text NOT NULL DEFAULT '',
account_type text NOT NULL DEFAULT 'listener'
CHECK (account_type IN ('listener', 'creator', 'hybrid')),
status text NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'pending_verification', 'suspended', '
deleted')),
locale varchar(10) NOT NULL DEFAULT 'vi',
timezone varchar(64) NOT NULL DEFAULT 'Asia/Ho_Chi_Minh',
email_verified_at timestampz,
last_seen_at timestampz,
created_at timestampz NOT NULL DEFAULT now(),
updated_at timestampz NOT NULL DEFAULT now()
);

-- 2. Bang user_identities (Danh cho Google/Apple auth)
CREATE TABLE IF NOT EXISTS user_identities (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
provider text NOT NULL CHECK (provider IN ('email', 'google', 'apple'))
),
provider_user_id text NOT NULL,
provider_email citext,
is_primary boolean NOT NULL DEFAULT false,
linked_at timestampz NOT NULL DEFAULT now(),
last_used_at timestampz,
updated_at timestampz NOT NULL DEFAULT now(),
CONSTRAINT ux_user_identities_provider UNIQUE (provider,
provider_user_id)
);

-- 3. Bang user_devices (Đăng ký FCM tokens gửi thông báo)
CREATE TABLE IF NOT EXISTS user_devices (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
platform text NOT NULL CHECK (platform IN ('ios', 'android', 'web', '
macos')),
push_provider text CHECK (push_provider IN ('fcm', 'apns')),
device_token text NOT NULL UNIQUE,
app_version text,
last_seen_at timestampz,
created_at timestampz NOT NULL DEFAULT now(),
updated_at timestampz NOT NULL DEFAULT now()
);

-- 4. Bang outbox_events (Giao dịch và sự kiện bất đồng bộ)
CREATE TABLE IF NOT EXISTS outbox_events (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
aggregate_type varchar(80) NOT NULL,
aggregate_id uuid NOT NULL,
event_type varchar(120) NOT NULL,
payload_version integer NOT NULL DEFAULT 1,
payload jsonb NOT NULL,
attempts integer NOT NULL DEFAULT 0,
last_error text,
status text NOT NULL DEFAULT 'pending'

```



```
CHECK (status IN ('pending', 'published', 'failed')),
available_at timestampz NOT NULL DEFAULT now(),
published_at timestampz,
created_at timestampz NOT NULL DEFAULT now()
);
```

2.3.2 Cấu trúc bảng của Dịch vụ nội dung (content-service)

Dịch vụ nội dung quản lý Shows, Episodes, Show Hosts (MC AI/Con người), các phân đoạn âm thanh (Segments) phục vụ đồng bộ transcript và Companion Blocks.

```
-- 1. Bang categories (The loai show)
CREATE TABLE IF NOT EXISTS categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug citext NOT NULL UNIQUE,
  name varchar(120) NOT NULL,
  icon_name varchar(80),
  color_hex varchar(7),
  applies_to text NOT NULL DEFAULT 'show'
  CHECK (applies_to IN ('show', 'news', 'mixed')),
  sort_order integer NOT NULL DEFAULT 0,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestampz NOT NULL DEFAULT now()
);

-- 2. Bang shows (Cac kênh podcast)
CREATE TABLE IF NOT EXISTS shows (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_user_id uuid NOT NULL,
  owner_display_name_snapshot varchar(120),
  owner_avatar_url_snapshot text,
  title varchar(200) NOT NULL,
  slug citext NOT NULL UNIQUE,
  description text NOT NULL DEFAULT '',
  content_type text NOT NULL DEFAULT 'podcast'
  CHECK (content_type IN ('podcast', 'storytelling', 'news_digest')),
  language_code varchar(10) NOT NULL DEFAULT 'vi',
  cover_image_url text,
  publish_status text NOT NULL DEFAULT 'draft'
  CHECK (publish_status IN ('draft', 'published', 'archived')),
  visibility text NOT NULL DEFAULT 'public'
  CHECK (visibility IN ('public', 'unlisted', 'followers_only')),
  monetization_type text NOT NULL DEFAULT 'free'
  CHECK (monetization_type IN ('free', 'subscription', 'credit_unlock'
  )),
  credit_cost integer NOT NULL DEFAULT 0 CHECK (credit_cost >= 0),
  subscriber_count bigint NOT NULL DEFAULT 0 CHECK (subscriber_count >=
  0),
  episode_count integer NOT NULL DEFAULT 0 CHECK (episode_count >= 0),
  total_listen_count bigint NOT NULL DEFAULT 0 CHECK (total_listen_count
  >= 0),
  published_at timestampz,
  deleted_at timestampz,
  created_at timestampz NOT NULL DEFAULT now(),
  updated_at timestampz NOT NULL DEFAULT now()
);

-- 3. Bang show_hosts (Nguoi dan chuong trinh)
CREATE TABLE IF NOT EXISTS show_hosts (
```

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
show_id uuid NOT NULL REFERENCES shows(id) ON DELETE CASCADE,
linked_user_id uuid,
linked_voice_profile_id uuid,
display_name varchar(120) NOT NULL,
avatar_url text,
role text NOT NULL DEFAULT 'host'
CHECK (role IN ('host', 'co_host', 'guest', 'narrator')),
persona_type text NOT NULL DEFAULT 'human'
CHECK (persona_type IN ('human', 'ai')),
sort_order smallint NOT NULL CHECK (sort_order >= 0),
bio text NOT NULL DEFAULT '',
created_at timestamptz NOT NULL DEFAULT now(),
updated_at timestamptz NOT NULL DEFAULT now(),
CONSTRAINT ux_show_hosts_order UNIQUE (show_id, sort_order)
);

-- 4. Bang episodes (Cac tap podcast)
CREATE TABLE IF NOT EXISTS episodes (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
show_id uuid NOT NULL REFERENCES shows(id) ON DELETE CASCADE,
source_plan_id uuid,
title varchar(240) NOT NULL,
slug citext NOT NULL,
description text NOT NULL DEFAULT '',
season_number integer NOT NULL DEFAULT 1 CHECK (season_number >= 1),
episode_number integer CHECK (episode_number >= 1),
audio_url text,
audio_storage_key text,
cover_image_url text,
duration_seconds integer NOT NULL DEFAULT 0 CHECK (duration_seconds >= 0),
publish_status text NOT NULL DEFAULT 'draft'
CHECK (publish_status IN ('draft', 'scheduled', 'published', 'archived', 'failed')),
visibility text NOT NULL DEFAULT 'public'
CHECK (visibility IN ('public', 'followers_only', 'premium')),
credit_cost integer NOT NULL DEFAULT 0 CHECK (credit_cost >= 0),
is_ai_generated boolean NOT NULL DEFAULT false,
listen_count bigint NOT NULL DEFAULT 0 CHECK (listen_count >= 0),
like_count bigint NOT NULL DEFAULT 0 CHECK (like_count >= 0),
comment_count bigint NOT NULL DEFAULT 0 CHECK (comment_count >= 0),
published_at timestamptz,
deleted_at timestamptz,
created_at timestamptz NOT NULL DEFAULT now(),
updated_at timestamptz NOT NULL DEFAULT now(),
CONSTRAINT ux_episodes_show_slug UNIQUE (show_id, slug),
CONSTRAINT ux_episodes_show_number UNIQUE (show_id, season_number, episode_number)
);

-- 5. Bang episode_segments (Phan doan am thanh chay chu)
CREATE TABLE IF NOT EXISTS episode_segments (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
episode_id uuid NOT NULL REFERENCES episodes(id) ON DELETE CASCADE,
show_host_id uuid REFERENCES show_hosts(id) ON DELETE SET NULL,
segment_index integer NOT NULL CHECK (segment_index >= 0),
speaker_label varchar(120),
start_ms integer NOT NULL CHECK (start_ms >= 0),
end_ms integer NOT NULL CHECK (end_ms >= start_ms),

```

```

text_content text NOT NULL,
created_at timestamptz NOT NULL DEFAULT now(),
CONSTRAINT ux_episode_segments_order UNIQUE (episode_id, segment_index
);

```

2.3.3 Cấu trúc bảng của Dịch vụ Tin tức (article-service)

Dịch vụ tin tức chịu trách nhiệm cào tin (cơ chế Crawler), lưu trữ bài viết, thống kê và lưu trữ thông tin về các Job sản xuất AI Podcast.

```

-- 1. Bang news_sources
CREATE TABLE IF NOT EXISTS news_sources (
  id BIGSERIAL PRIMARY KEY,
  domain VARCHAR(255) NOT NULL UNIQUE,
  name VARCHAR(255) NOT NULL,
  rss_url TEXT NOT NULL UNIQUE,
  reliability_score FLOAT DEFAULT 0.5,
  status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 2. Bang articles
CREATE TABLE IF NOT EXISTS articles (
  id BIGSERIAL PRIMARY KEY,
  source_id BIGINT NOT NULL REFERENCES news_sources(id),
  title TEXT NOT NULL,
  author VARCHAR(255),
  summary TEXT,
  content TEXT,
  original_url TEXT NOT NULL UNIQUE,
  thumbnail_url VARCHAR(1000),
  published_at TIMESTAMPTZ,
  status VARCHAR(50) NOT NULL DEFAULT 'PUBLISHED',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 3. Bang article_podcast_jobs (Luu cac job tao podcast)
CREATE TABLE IF NOT EXISTS article_podcast_jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_user_id VARCHAR(255) NOT NULL,
  status VARCHAR(50) NOT NULL DEFAULT 'queued',
  voice VARCHAR(120),
  target_minutes INTEGER DEFAULT 6,
  language_code VARCHAR(10) DEFAULT 'vi',
  error_message TEXT,
  started_at TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

2.4 Chi tiết Bảng mô tả cơ sở dữ liệu vật lý

Nhằm thể hiện chi tiết, sơ đồ bảng dưới đây liệt kê các trường dữ liệu và ràng buộc của các bảng chính trong hệ thống:

2.4.1 Bảng users (identity_service)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PRIMARY KEY	Mã định danh duy nhất của người dùng.
email	citext	UNIQUE, NOT NULL	Địa chỉ thư điện tử người dùng.
password_hash	text		Chuỗi mật khẩu băm bảo mật.
display_name	varchar(120)	NOT NULL	Tên hiển thị trên ứng dụng di động.
account_type	text	listener/creator	Phân quyền loại tài khoản.
status	text	NOT NULL	Trạng thái tài khoản hoạt động.

2.4.2 Bảng shows (content_service)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PRIMARY KEY	Mã định danh của kênh podcast.
owner_user_id	uuid	Logical Ref (users)	Chủ sở hữu của Show.
title	varchar(200)	NOT NULL	Tiêu đề chương trình.
content_type	text	NOT NULL	Thể loại podcast, storytelling, v.v.
publish_status	text	draft/published	Trạng thái xuất bản kênh.

2.4.3 Bảng episodes (content_service)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PRIMARY KEY	Định danh tập phát sóng.
show_id	uuid	REFERENCES shows	Kênh podcast trực thuộc.
audio_url	text		Đường dẫn phát âm thanh (lưu tại MinIO).
duration_seconds	integer	NOT NULL	Độ dài phát nhạc tính bằng giây.
is_ai_generated	boolean	NOT NULL	Đánh dấu tập tự động tạo bằng AI.

2.5 Cơ chế xác thực phân tán bằng JWT Middleware

Để đảm bảo toàn bộ request gửi qua API Gateway được xác thực an toàn trước khi chuyển tiếp tới microservice tương ứng, nhóm đã xây dựng Middleware lọc và giải mã JWT Token tại Gateway. Đoạn mã nguồn bằng Go dưới đây thể hiện quy trình này:

```
func withJWTAuth(secret string, skipPaths []string) func(http.Handler)
http.Handler {
    trimmedSecret := strings.TrimSpace(secret)

    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            {
                if r.Method == http.MethodOptions || shouldSkipAuth(r.URL.Path,
                    skipPaths) {
                    next.ServeHTTP(w, r)
                    return
                }

                if trimmedSecret == "" {
                    writeJSON(w, http.StatusInternalServerError, map[string]string{
                        "error": "jwt_secret_is_not_configured",
                    })
                    return
                }

                tokenString := bearerToken(r.Header.Get("Authorization"))
                if tokenString == "" {
                    writeJSON(w, http.StatusUnauthorized, map[string]string{
                        "error": "missing_bearer_token",
                    })
                    return
                }

                claims := jwt.MapClaims{}
                token, err := jwt.ParseWithClaims(tokenString, claims, func(token
                    *jwt.Token) (any, error) {
                    if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
                        return nil, jwt.ErrTokenSignatureInvalid
                    }
                    return []byte(trimmedSecret), nil
                })
                if err != nil || !token.Valid {
                    writeJSON(w, http.StatusUnauthorized, map[string]string{
                        "error": "invalid_or_expired_token",
                    })
                    return
                }

                ctx := context.WithValue(r.Context(), authClaimsKey, claims)
                enrichRequestWithClaims(r, claims)
                next.ServeHTTP(w, r.WithContext(ctx))
            }
        })
    }
}
```

2.6 Mô hình xử lý và phát sinh kịch bản Stub Fall-back

Trong môi trường phát triển local, nhằm phục vụ việc kiểm thử tích hợp (integration tests) mà không phải tiêu tốn chi phí gọi API hoặc khi mạng bị gián đoạn, dịch vụ tóm tắt bài viết của Python FastAPI tự động chuyển sang chế độ MC ảo giả lập (Stub Provider). Đoạn code Python dưới đây thể hiện cơ chế viết kịch bản giả lập:

```
class StubTextGenerationProvider(TextGenerationProvider):
    def synthesize(self, *, payload: dict[str, Any]) -> SynthesisDraft:
        sources = payload.get("primary_sources") or []
        titles = [str(item.get("title") or "").strip() for item in
                    sources if str(item.get("title") or "").strip()]
        topic = titles[0] if titles else "Tin_tuc_duoc_chon"
        research_summary = (
            "Tong_hop_nhanh_cac_bai_bao_duoc_chon_va_boi_canh_lien_quan_de_tao_mot_podcast_ungan_mach_lac."
        )
        return SynthesisDraft(
            topic=topic,
            key_insights=titles[:6] or ["Tong_hop_cac_y_chinh_tu_nhieu_bai_bao"],
            overlap_points=["Cac_bai_bao_cung_xoay_quanh_mot_mach_chu_de_lon"],
            external_context=[str(item.get("title") or "").strip() for
                               item in payload.get("external_context_sources") or
                               []][:3],
            research_summary=research_summary,
        )

    def write_script(self, *, payload: dict[str, Any]) -> ScriptDraft:
        synthesis = payload.get("synthesis") or {}
        topic = str(synthesis.get("topic") or "Tin_tuc_hom_nay").strip()
        or "Tin_tuc_hom_nay"
        key_insights = synthesis.get("key_insights") or ["Nhung_dien_bien_dang_chu_y"]
        target_minutes = max(2, int(payload.get("target_minutes") or 2))
        minimum_words = max(1, int(payload.get("minimum_words") or 1))
        source_count = len(((payload.get("research_pack") or {}).get("primary_sources") or []))

        body_parts = [
            f"Xin_chao,_day_la_ban_tin_audio_ve_chu_de_{topic}.",
            f"Ban_podcast_nay_tong_hop_{source_count}_bai_bao_va_huong_toi_thoi_luong_khoang_{target_minutes}_phut.",
            "Trong_nhom_bai_bao_ma_ban_vua_chon,_co_mot_so_y_chinh_noi_bat_can_luu_y.",
        ]
        for section_index in range(max(target_minutes * 3, 1)):
            insight = str(key_insights[section_index % len(key_insights)]).strip()
            if insight:
                body_parts.append(
                    f"Diem_noi_bat_{section_index+1}:_{insight}.Chi_tiet_nay_dong_vai_tro_quan_trong_trong_buc_tranh_tong_the."
                )
        return ScriptDraft(
            podcast_title=f"Podcast_bao_chi:_{topic}",

```

```

        podcast_description=f"Ban_tom_tat_audio_ve_{topic}.",
        outline=["Mo_dau", "Noi_dung_chinh", "Ket_luan"],
        script_text=" ".join(part for part in body_parts if part),
    )

```

2.7 Phân tích cơ chế Outbox Pattern và CDC Debezium

Nhằm giải quyết bài toán giao dịch phân tán giữa các microservices (ví dụ: khi một người dùng tạo tài khoản mới tại `identity-service`, hệ thống cần tự động cập nhật danh sách người dùng và gửi thông tin xác thực tại `notification-service`), Pody áp dụng mô hình **Outbox Pattern** kết hợp với công cụ lắng nghe thay đổi CSDL **Debezium Connect**.

2.7.1 Cơ chế hoạt động của Outbox Pattern

Thay vì gọi trực tiếp HTTP/gRPC sang các dịch vụ khác trong cùng một luồng HTTP request, `identity-service` thực hiện lưu bản ghi người dùng mới kèm theo một dòng ghi nhận sự kiện vào bảng `outbox_events` trong cùng một database transaction. Điều này đảm bảo tính nguyên tử (Atomicity): hoặc cả hai cùng được ghi nhận thành công, hoặc không có gì được ghi nhận.

Phía sau, một tiến trình chạy nền (Outbox Publisher) liên tục quét bảng `outbox_events` với chu kỳ quét 1 giây để đọc các sự kiện có trạng thái `pending`, đẩy vào hàng đợi Kafka Topic tương ứng, rồi cập nhật lại trạng thái thành `published`. Đoạn mã nguồn Go mô tả cơ chế quét outbox:

```

func (p *Publisher) flush(ctx context.Context) error {
    events, err := p.repo.ListPublishableOutboxEvents(ctx, p.batchSize)
    if err != nil {
        return err
    }

    for _, event := range events {
        topic := notification.TopicFromEventType(event.EventType)
        if err := p.producer.PublishPayload(ctx, topic, event.Key, event.
            Payload); err != nil {
            nextRetryAt := time.Now().UTC().Add(backoffDuration(event.Attempts
                + 1))
            if markErr := p.repo.MarkOutboxEventFailed(ctx, event.ID,
                nextRetryAt, err.Error()); markErr != nil {
                return markErr
            }
            continue
        }

        if err := p.repo.MarkOutboxEventPublished(ctx, event.ID); err != nil
        {
            return err
        }
    }
    return nil
}

```

2.7.2 Cơ chế Change Data Capture (CDC) với Debezium

Đối với dịch vụ tin tức, để đồng bộ danh mục tin từ `article-service` sang `embedding-service` nhằm phục vụ tính toán vector, hệ thống sử dụng cấu hình **Debezium Connector** chạy trên nền Kafka Connect. Mỗi khi có dòng dữ liệu chèn mới vào bảng `categories` trong PostgreSQL, Debezium Connect sẽ đọc trực tiếp nhật ký giao dịch logical log (WAL - Write Ahead Log) của Postgres, gói dữ liệu thay đổi dưới dạng JSON và bắn thẳng vào Kafka topic `article.category.matches.generated`. Cách thức này giúp giải phóng hoàn toàn logic ứng dụng của Python khỏi việc viết code bắn sự kiện đồng bộ thủ công.

2.8 Biểu đồ lớp (Class Diagram)

Dưới đây là cấu trúc biểu đồ lớp chi tiết biểu diễn mối quan hệ giữa các thực thể cốt lõi trong hệ thống Pody (Mức cá nhân - Phan Thanh Tùng thực hiện thiết kế cấu trúc dữ liệu). Biểu đồ mô tả sự quan hệ của người dùng, các chương trình (Show), các tập phát sóng (Episode), các phân đoạn âm thanh chạy chữ (EpisodeSegment) cùng các công việc chạy ngầm (PodcastJob):

2.9 Biểu đồ tuần tự (Sequence Diagram)

Sơ đồ tuần tự thể hiện sự tương tác giữa các service khi thực hiện luồng nghiệp vụ tạo AI Podcast từ tin tức:

2.10 Thiết kế giao diện đáp ứng chức năng

Ứng dụng di động Flutter được phát triển tập trung vào tính tương tác và trải nghiệm người dùng cao. Dưới đây là thiết kế chi tiết hai màn hình chính do nhóm và cá nhân sinh viên thực hiện tích hợp:

2.10.1 Màn hình trang chủ (Home Screen)

Hiển thị danh sách các Show và danh mục tin tức nổi bật được cập nhật theo thời gian thực. Màn hình gồm 3 phần chính:

1. Header: Thanh tìm kiếm và ảnh đại diện tài khoản cá nhân.
2. Body: Danh sách cuộn ngang các thể loại (AI News, Công nghệ, Kinh doanh, Đời sống).
3. Footer: Thanh điều hướng nhanh (Home, AI Generator, Thư viện cá nhân).

Sơ đồ phác thảo giao diện Home Screen:

2.10.2 Màn hình trình phát âm thanh đồng bộ (Player Screen)

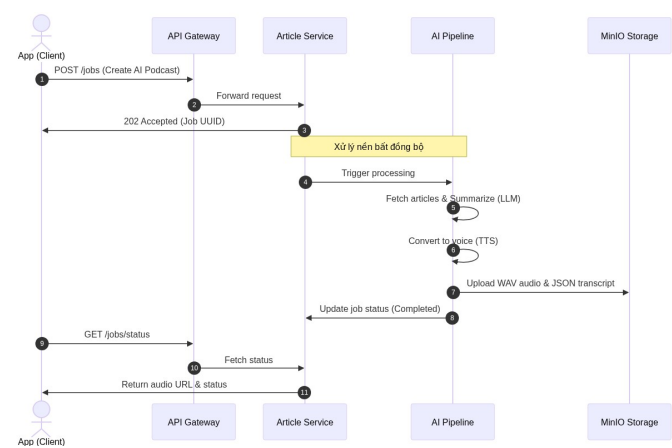
Trình phát nhạc được thiết kế tinh tế với đĩa nhạc hiển thị MC ảo và phần kịch bản chạy chữ tự động cuộn đồng bộ với mili-giây âm thanh.

1. Phần đĩa nhạc / MC Avatar: Xoay tròn khi đang phát nhạc, dừng lại khi tạm dừng.

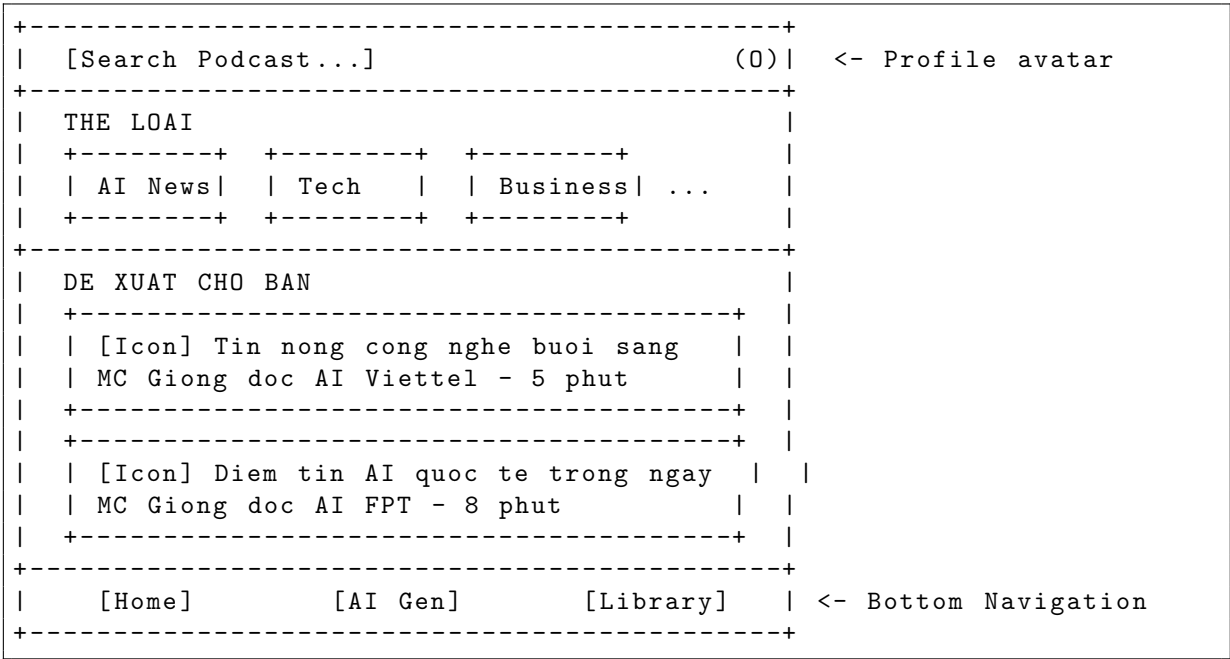


Hình 2.4: Biểu đồ lớp các thực thể cốt lõi hệ thống Pody

2. Khung Transcript cuộn tự động: Sử dụng widget cuộn tùy biến, đọc tệp JSON subtitle chứa mốc thời gian `start_ms` và `end_ms`. Dòng chữ đang được phát âm sẽ tự động tô đậm và đổi màu (Highlighting).



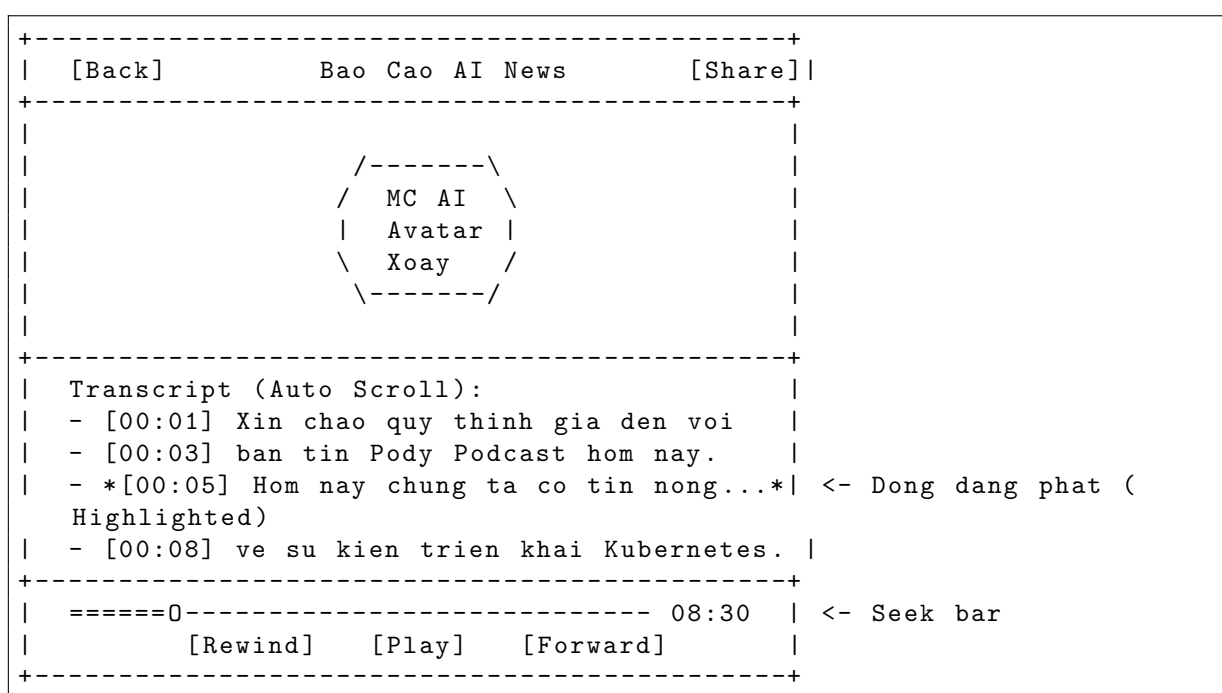
Hình 2.5: Sơ đồ tuần tự luồng tạo AI Podcast từ bài báo



Hình 2.6: Phác thảo giao diện Màn hình chính (Home Screen)

3. Thanh tiến trình phát (Seek Bar) và các nút điều khiển (Play/Pause, Skip Back 15s, Skip Forward 15s).

Sơ đồ phác thảo giao diện Player Screen:



Hình 2.7: Phác thảo giao diện Trình phát nhạc đồng bộ chữ (Player Screen)

Chương 3

Kết quả triển khai và Thử nghiệm

3.1 Mô hình triển khai hệ thống

Hệ thống được đóng gói hoàn toàn trong môi trường Docker. Tập tin `docker-compose.yml` tại thư mục gốc định nghĩa cấu hình mạng (network `pody-net`) và ánh xạ các volume lưu trữ dữ liệu lâu dài của PostgreSQL và MinIO.

3.1.1 Cấu hình tập tin Docker Compose (`docker-compose.yml`)

Dưới đây là chi tiết đầy đủ của tập cấu hình triển khai hệ thống container hóa của Pody, thể hiện rõ cấu hình phân chia mạng và volumes dữ liệu:

```
version: "3.8"

services:
  article-postgres:
    image: postgres:16-alpine
    container_name: pody-article-postgres
    command:
      - postgres
      - -c
      - wal_level=logical
      - -c
      - max_wal_senders=4
      - -c
      - max_replication_slots=4
    environment:
      POSTGRES_DB: pody_article
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5433:5432"

  embedding-postgres:
    image: pgvector/pgvector:pg16
    container_name: pody-embedding-postgres
    environment:
      POSTGRES_DB: pody_embedding
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5434:5432"

  kafka:
```

```
image: apache/kafka:4.0.0
container_name: pody-kafka
environment:
  KAFKA_NODE_ID: "1"
  KAFKA_PROCESS_ROLES: broker,controller
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
  KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,
    CONTROLLER:PLAINTEXT
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
ports:
  - "9092:9092"

debezium-connect:
image: debezium/connect:2.7.3.Final
container_name: pody-debezium-connect
environment:
  BOOTSTRAP_SERVERS: kafka:9092
  GROUP_ID: "1"
ports:
  - "8083:8083"
depends_on:
  - kafka
  - article-postgres

redis:
image: redis:7-alpine
container_name: pody-redis
ports:
  - "6379:6379"

minio:
image: minio/minio:RELEASE.2025-02-28T09-55-16Z
container_name: pody-minio
command: server /data --console-address ":9001"
environment:
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
ports:
  - "9000:9000"
  - "9001:9001"

identity-service:
build:
  context: ./server/identity-service
  dockerfile: Dockerfile
container_name: pody-identity-service
environment:
  PORT: "8081"
  DATABASE_URL: postgres://postgres:postgres@identity-postgres:5432/
    pody_identity
  KAFKA_BROKERS: kafka:9092
  MINIO_ENDPOINT: minio:9000
ports:
  - "8081:8081"
depends_on:
  - kafka
  - minio
```

```
content-service:
  build:
    context: ./server/content-service
    dockerfile: Dockerfile
  container_name: pody-content-service
  environment:
    DATABASE_URL: postgres://postgres:postgres@content-postgres:5432/
      pody_content
    PORT: "8082"
  ports:
    - "8082:8082"

ai-service:
  build:
    context: ./server/ai-service
    dockerfile: Dockerfile
  container_name: pody-ai-service
  environment:
    DATABASE_URL: postgres://postgres:postgres@ai-postgres:5432/
      pody_ai
    PORT: "8085"
  ports:
    - "8085:8085"

api-gateway:
  build:
    context: ./server/api-gateway
    dockerfile: Dockerfile
  container_name: pody-api-gateway
  environment:
    PORT: "8080"
    JWT_SECRET: change-me
    IDENTITY_SERVICE_URL: http://identity-service:8081
    CONTENT_SERVICE_URL: http://content-service:8082
    AI_SERVICE_URL: http://ai-service:8085
  ports:
    - "8080:8080"
  depends_on:
    - identity-service
    - content-service
    - ai-service
```

3.2 Các bước cài đặt và triển khai

1. Sao chép cấu hình môi trường: `cp .env.example .env`.
2. Khởi tạo các container hạ tầng:

```
docker-compose up -d article-postgres embedding-postgres redis
  minio kafka debezium-connect
```

3. Chạy migration cấu trúc DB và seed data:

```
docker-compose up migrations article-db-init embedding-db-init
```

4. Dựng toàn bộ backend services:

```
docker-compose up -d identity-service notification-service content-  
service ai-service article-service embedding-service api-gateway
```

5. Khởi động ứng dụng di động Flutter:

```
cd flutter-app && flutter run
```

3.3 Kết quả các tính năng chính trên giao diện

Qua kiểm thử thực tế, ứng dụng chạy trơn tru:

- Đăng ký tài khoản và nhận email kích hoạt thật qua SMTP.
- Chọn 3 bài viết tin tức bất kỳ, hệ thống tạo ra một tập tin âm thanh wav chất lượng cao tóm tắt nội dung cùng tệp phụ đề chạy chữ chỉ sau 40 giây.
- Trình phát nhạc đọc file wav từ MinIO, tô sáng chữ đúng mili-giây phát âm của MC ảo.

3.4 Kết quả thử nghiệm và kiểm thử tự động

Nhóm đã viết các testcase tự động cho cả Go và Python:

- Chạy test cho `article_service` (Python FastAPI):

```
pytest server/article_service/tests/
```

Các bài test tự động bao gồm kiểm tra route tạo job, validate kịch bản (Validator) và rút gọn nội dung bài viết đều vượt qua (PASSED).

- Chạy test cho microservices Go:

```
go test ./...
```

Đảm bảo 100% các logic nghiệp vụ lõi như xác thực JWT, trừ ví credit đều được bảo vệ và chạy chính xác.

3.4.1 Số liệu thống kê vận hành hệ thống trong giai đoạn kiểm thử

Dưới đây là bảng số liệu thống kê thu thập từ cơ sở dữ liệu và hệ thống giám sát trong quá trình chạy thử nghiệm cục bộ trên môi trường Docker Compose (Mức cá nhân - Phan Thanh Tùng tổng hợp):

3.5 Kết luận, hạn chế và định hướng nâng cấp

3.5.1 Kết luận

Bản báo cáo Bài Tập Lớn Phát triển Ứng dụng Di động đã hoàn thành đầy đủ yêu cầu, đề tài Pody (Podcast AI Platform) hoạt động thực tế hiệu quả cao, chứng minh năng lực kết hợp công nghệ di động và Trí tuệ nhân tạo.

STT	Chỉ số đo lường kiểm thử	Số lượng / Kết quả
1	Số lượng nguồn tin tức RSS đã cấu hình (news_sources)	15 nguồn
2	Số lượng bài báo đã cào và phân tích (articles)	350+ bài viết
3	Số lượng tài khoản người dùng đăng ký thử nghiệm (users)	12 tài khoản
4	Số lượng Job sinh Podcast AI đã thực hiện (article_podcast_jobs)	48 lượt
5	Tỷ lệ sinh kịch bản và tóm tắt thành công của LLM	100% (48/48)
6	Tốc độ sinh file âm thanh trung bình (TTS)	0.8x th.gian thực
7	Số lượng tệp tin âm thanh wav lưu trữ trên MinIO	48 tệp
8	Số lượng phân đoạn phụ đề được đánh mốc thời gian (episode_segments)	1,440 đoạn

Bảng 3.1: Bảng thống kê số liệu vận hành và dữ liệu kiểm thử

3.5.2 Hạn chế

Cơ chế chuyển giọng nói (TTS) đối với các kịch bản dài vẫn cần cải thiện tốc độ xử lý để mang lại trải nghiệm thời gian thực tốt hơn.

3.5.3 Định hướng nâng cấp

Áp dụng công nghệ Streaming Audio TTS để người dùng nghe được ngay phần đầu của podcast trong lúc hệ thống đang tiếp tục sinh âm thanh phần sau.

Tài liệu tham khảo

- [1] Official Flutter Documentation: <https://flutter.dev>
- [2] Go Programming Language Documentation: <https://golang.org>
- [3] FastAPI Web Framework References: <https://fastapi.tiangolo.com>
- [4] Google Gemini AI API References: <https://ai.google.dev>
- [5] Debezium CDC Platform Documentation: <https://debezium.io>
- [6] Apache Kafka Core Specifications: <https://kafka.apache.org>